

Claude Code 的四种循环：别再当 Agent 的人肉开关 已付费

原创 克劳德猎手 克劳德猎手 2026年7月8日 11:17 山东 [听全文](#)

前言

从手动发 Prompt 到设计自治系统，Claude Code 的用法正在发生根本变化。这篇文章把四种循环模式讲透了：轮次、目标、时间、主动——每多一层，你交给系统的东西就多一层。读完你会知道什么任务该用哪种循环，以及怎么设定一个机器能自己验证的停止条件。

本文收录于「[Claude 官方精读](#)」合集

Claude Code 的使用方式正在发生一个根本性的变化。**核心竞争力不再是写出完美的 Prompt，而是选择合适的循环模式，给它一个机器可以自行验证的停止条件。**这篇文章来自 clauddefa.st 的完整指南，我读完后的判断是：循环工程（Loop Engineering）会成为和 Prompt Engineering 同等重要的基础技能，但它的门槛更高、回报也更大。

每一条发出去的 Prompt，其实都在跑一个循环。Claude 评估上下文、调用工具、拿到结果、再评估——这个「[Agent Loop](#)」是内建的引擎，你不需要管它，也管不了它。

但这篇文章讲的是另一个层面的东西：**你围绕这个引擎设计的用户面循环。**什么时候触发、什么时候停、跑多少轮。四种模式，每种适合完全不同的场景。

理解这件事之后，你会发现大部分 Claude Code 用户的日常用法都停在第一层：手动发 Prompt，手动判断做完没，手动决定下一步。而真正拉开效率差距的，是能不能**把这些判断交给系统**。

举个例子。`/goal` 这个命令，大部分人的用法是「帮我优化首页性能」。但 `/goal` 真正的用法是：「把首页 Lighthouse 分数拉到 90 以上，最多试 5 次」。前者 Claude 改了三个变量名就停了，后者它在两小时内跑了 5 轮迭代、换了三种方案。

区别在哪？区别在于你有没有把「做完」写成一个 Claude 可以读到的事实，写成它不需要猜测的**终点**。这是整篇文章最核心的洞察，也是四种循环模式的底层逻辑。

继续往下读，你会得到：

一份循环选择速查表：四类循环（轮次循环、目标循环、时间循环、主动循环）的触发机制、停止条件、适用场景和成本控制策略，一张表对照。不需要记住所有命令，看一眼就知道手头的任务该用哪种。

5 类可量化停止条件模板：覆盖测试、[Lighthouse](#)、lint、队列、PR 五种最常见场景，拿过去改个数字就能用。附带弱条件 vs 强条件的逐条对比，以及我自己踩过的一個典型坑：设了「让代码更清晰」的条件，Claude 改了三个变量名就宣布完成。改成「eslint 零 warning + 圈复杂度 ≤10

+ 文件 ≤300 行」之后，它实打实重构了两小时。把「感觉」翻译成「数字」，这篇文章会告诉你每一步怎么做。

质量保持和成本控制的完整策略：四个习惯防止代码在反复迭代中烂掉（包括怎么用第二个 Agent 做独立审查，而不是让同一个 Agent 自己写代码自己查），六条成本策略从选模型到设轮询间隔，外加 /usage、/goal、/workflows 三个诊断命令的实操用法。

这不是官方文档的翻译。官方文档告诉你 /goal、/loop、/schedule 这几个命令存在。但它不会告诉你怎么把一个模糊的「做好了」翻译成机器可以验证的数字，也不会告诉你什么场景上循环反而亏本：上错了烧的 token 比省的时间还多，而且一个写好验证 Skill 的轮次循环有时候比 /goal 更划算。这些判断来自社区实践者跑了几百次循环烧出来的经验，你在别处读不到。读完你能直接写出第一个可工作的循环，条件不会写太模糊、轮次上限不会设太高，绕过新手必踩的那几个坑。

1 Agent Loop 和 Loops，不是一回事

文章一开头就做了一个很重要的区分，值得花一段讲清楚。两个概念共享「Loop」这个词，但指的是完全不同的东西。

Agent Loop（单数）是 Claude 内部执行引擎的运转方式：收到 prompt → 评估当前状态 → 调用工具 → 收到工具返回 → 再评估 → 重复直到完成。这是内建的、自动的，你发任何一条 Prompt 它都在跑这个循环。可以把它理解成发动机——不管你开什么车、走什么路，发动机的工作原理是一样的。

Loops（复数）是你围绕这个引擎设计的使用模式：怎么触发的、什么时候停、跑多少轮。这是你可以主动选择和配置的东西。你可以设计一个循环让它每 5 分钟检查一次 PR 状态，也可以设计一个循环让它不达目标不罢休。

我见过不少开发者把 /loop 当成「Claude 内部改了什么机制」。其实没改——发动机还是那台发动机。改的是你的驾驶方式。理解这一点之后，再看四种循环模式就清楚了：发动机没换，换的是驾驶方式。四种循环就是四种驾驶模式。

2 Loop Engineering：从敲命令到设计系统

文章用了一个我特别喜欢的词来描述这个转变：**Loop Engineering（循环工程）**。你不再是在终端里一条一条敲命令，而是在设计一个系统——这个系统会自动触发 Agent、自动判断完成度、自动决定要不要再来一轮。

核心洞察是这句：**一旦 Agent 能自己验证自己的工作，瓶颈就从「你打字多快」变成了「你把『做完』定义得多清楚」。**

一个靠人驱动的会话，你在循环里。每轮都要你看一眼结果、判断行不行、决定下一步干嘛。一个设计良好的循环把你从循环里拿出来——但这只有在 Agent 能检查你本来会检查的那些东西时才安全。

不是每个任务都需要上循环。文章给了一个简单判断框架：任务重复吗？任务跑得长吗？任务有可衡量的终点吗？至少两个答案是「是」，才值得设计循环。我自己的习惯更简单：先问「这事需要我盯着判断吗」。如果不需要，直接考虑用哪种循环。

3 四种循环，逐类拆解

文章把用户面循环分成四类，按「你交出去的是什么」从少到多排列。下面逐一展开，每一类我都会给出原文的核心逻辑加上自己的使用判断。

轮次循环 (Turn-Based Loops)

这是最基础的模式，每个人都在用。触发方式是你发一条 Prompt，停止条件是 Claude 自己判断做完了或者需要你给更多信息。适用于一次性的、短小的、不在日程上的工作。

大部分人停留在这个层面。问题不在于「用了轮次循环」——问题在于**每一轮都要你来验证**。你点了发送之后等着 Claude 跑完，然后自己打开浏览器检查、自己读代码、自己判断行不行。你还是瓶颈。

文章的升级建议：**把验证逻辑编码进 Skill**，给 Claude 一套可以自己执行的检查步骤。原文给的例子是一个前端改动验证 Skill：

```
.claude/skills/verify-frontend-change/SKILL.md
```

1. 启动 dev server，打开修改后的页面
 2. 交互验证：点击控件，确认状态变化
 3. 截前后对比截图
 4. 检查浏览器控制台，确认零新增错误或警告
 5. 跑一次性能 trace，审计 Core Web Vitals
- 任一步失败则修复后从第 1 步重新开始

这个 Skill 本质上就是把「你打开浏览器看的那一眼」变成了「Claude 可以自己执行的一个脚本」。你不再需要盯着看，验证变成了循环内建的一个环节。任何一步失败就修复后重新从第一步开始。这个「修完重跑」的逻辑也是 Skill 自带的，不用你手动触发。

目标循环 (Goal-Based Loops, /goal)

这是四种循环里我目前用得最多的一种。机制是这样的：你给定一个目标和轮次上限，每次 Claude 想停下来的时候，**一个独立的 evaluator model 会检查你设的条件**。条件不满足就把它踢回去继续干，直到目标达成或者达到轮次上限。

关键词是「独立的 evaluator」。让 Claude 自己判断自己做完了没，跟让学生自己给自己打分没区别。而是一个外部评估模型，带着你给的条件去检查 Claude 的输出。这层独立的「裁判」是 /goal 跟轮次循环最本质的区别。

文章里举的例子很典型：

```
shell
```

```
/goal 把首页 Lighthouse 分数拉到 90 以上, 最多试 5 次
```

注意这个目标是怎么写的。「90 或以上」是一个数字,「最多 5 次」是一个明确的硬上限。没有歧义, Claude 和 evaluator 都能准确理解。轮次上限不是摆设——不设上限的话, /goal 可能在两个方案之间反复横跳到 token 耗尽。我一般设 5~8 次, 既能给 Claude 足够的尝试空间, 又不会让它无限消耗 token。

/goal 最适合那种「你知道什么算好, 但不确定怎么做到」的任务。性能优化、测试覆盖率提升、代码重构, 目标可以是数字 (分数、覆盖率), 也可以是状态 (lint 零报错、PR 已合并)。

时间循环 (Time-Based Loops, /loop + /schedule)

时间循环按触发方式分成两种, 区别不止于本地还是云端。

/loop 在你的本地机器上跑, 按定时器触发。关掉电脑循环就停了。适合快速轮询:「每 5 分钟检查我的 PR 有没有新的 review comments, 有就处理」。不写间隔的话 Claude 会自己掌控节奏——做完一轮再决定要不要马上开始下一轮。

```
shell
```

```
/loop 5m 检查我的 PR, 处理 review 意见, 修复失败的 CI
```

/schedule (research preview) 把循环搬到 Anthropic 的云端基础设施上跑。cron 定时、API 调用、GitHub webhook 都能触发。你关了电脑、睡了觉, 它还在跑。两者的本质区别是: **本地 /loop** 是你上班时的助手, **/schedule** 是你下班后的夜班工人。

文章还提了一个 Monitor tool, 做事件驱动唤醒: 只在被监控命令有输出时才激活会话。这比定时轮询省 token 得多, 但也有局限: 需要先知道监控什么命令。

主动循环 (Proactive Loops)

这是前面三种的组合升级版。把 **/schedule** 当触发器 (按 cron 跑)、**/goal** 定义做完标准、dynamic workflows (也是 research preview) 编排多个子代理并行处理。

一个典型场景: 每小时检查项目反馈频道的 bug 报告 → 每个 bug 用 **/goal** 跑一轮分类和回复 → 需要修代码的 bug 用 dynamic workflow 在独立 worktree 里并行修 → 修完由对抗式审查 agent 验证。这和 [2.1.202 那篇聊的工作流可观测性](#) 是同一个方向: 当多个 agent 并行跑的时候, 需要知道谁在摸鱼、谁卡住了。

成本控制策略很关键: 常规分类、回复用 Haiku 级别的模型, 代码修复用 Sonnet, 最终的审查判断用 Opus。不同难度的任务匹配不同成本的模型, 而不是所有任务都用同一个模型从头发动机。

不过说实话, 主动循环目前还是愿景大于现实。dynamic workflows 和 **/schedule** 都还在 research preview, 稳定性不够。如果你现在就想在生产环境跑, 最靠谱的方案是 **/loop** + **/goal** + 一个写得好的验证 Skill, 在自己机器上跑, 出了问题你能第一时间感知到。

4 停止条件：循环的生与死

这是全文含金量最高的一节。前面的四种循环模式是「怎么跑」，停止条件是「什么时候停」。停得太早等于白跑，停得太晚等于空耗 token。

核心规则一句话：**让停止条件成为 Claude 可以测量的事实，写成它不需要猜测的终点。**

文章给了一组对比，我到现在都记得：

✗ 弱条件	✓ 强条件
make the tests better	every test in the suite passes
让代码更清晰	eslint 零 warning + 圈复杂度 ≤10

左边那一列，没有一行是 Claude 可以检查的。什么叫「好」？什么叫「清晰」？Claude 只能猜。右边那一列，每条都是 Claude 能直接读到的事实——跑一遍测试套件读退出码、跑一遍 lint 看输出。

文章列出了可靠的可量化停止条件的几个特征：测试全部通过、Lighthouse 分数达到或超过 X、lint 零报错、队列零条目、PR 状态为 merged。「**每一条都是 Claude 可以读到的事实。数字和状态不需要「感觉」。**」

我自己踩过一个典型的坑。有次设了个条件叫「让代码更清晰」，Claude 改了三个变量名就停了。从它的角度看，它确实做了让代码更清晰的事。后来我改成「eslint 零 warning + 所有函数圈复杂度 ≤10 + 所有文件 ≤300 行」，它实打实重构了两小时，拆了四个大函数、去掉了两处重复逻辑。区别就是：前者需要 Claude 自己判断什么叫「清晰」，后者给了它三把可以量化的尺子。

5 循环跑起来了，质量怎么保住

循环会让 Agent 反复修改代码。改得越多，出问题的概率越大。文章给了四个保持质量的习惯，每一条展开说：

第一，保持代码库整洁。Claude 会遵循现有代码的模式。如果现有代码已经是一团乱，循环跑出来的东西只会更乱。重构和循环不要同时做，先把底子收拾干净再上循环。

第二，给 Claude 验证自己工作的手段。把「好」编码成 Skill，包含具体的检查步骤和验证命令。每次循环迭代结束后自动跑一遍验证 Skill，就像 CI pipeline 里自动跑测试一样。这个 Skill 不是写一次就完了。每踩一个坑就加一条检查规则，时间越长 Skill 越强。

第三，确保文档可达。模型训练数据里的 API 可能已经过时了。每次跑循环前把相关框架的最新文档拉进上下文，比让模型凭记忆猜要可靠得多。这一点在长循环里尤其关键：第一轮猜错了可能后面每一轮都基于错误的假设在跑。

第四，用第二个 Agent 做审查。同一个 Agent 既写代码又审查自己，跟请室友帮你 proofread 论文差不多，结构上的问题它自己也看不出来。 `/code-review` 和 Code Review for GitHub 可以充当这个「第二双眼睛」。和 [图 2.1.199 那次子代理报错被吞的修复](#) 一样。验证机制必须独立于生成机制，否则形同虚设。

文章里有一句话我特别认同，直接引原意：**当结果不达标时，把修复编码进系统，让以后的每次迭代都不会再踩同一个坑。一个会升级自己的循环，比一个只会跑的循环有价值得多。**

6 Token 用量：循环的隐形成本

循环跟单次 Prompt 最大的区别是：循环会持续消耗 token，不管有没有产出。你以为它跑了 5 轮在干活，但第 3 轮和第 4 轮可能在做无效探索，每次探索都在烧 token。

文章给了几个具体的成本控制策略，我按重要性排了序：

选对模型和模式。三行代码的改动不需要 `/goal` + Opus，一条 Prompt 用 Sonnet 就能解决。小任务用重模式是循环成本的最大来源。

先跑小规模试点。不确定一个循环设计行不行？先拿一个文件、一个组件跑一轮，看看 token 消耗量和产出质量，再决定要不要全量跑。这跟做 A/B 测试一个道理：在小流量上验证假设，别一上来就全量。

确定性操作用脚本，别让模型推理。批量重命名文件、格式化代码、查 API 文档，这些用 Shell 脚本或 grep 就能搞定，比让 Claude 推理每一步便宜几十倍。循环里混入确定性操作是纯浪费。

轮询间隔要匹配现实。`/loop 1m` 检查一个每天只变一次的数据源，跟 `/loop 30m` 检查一个每分钟都可能新消息的频道，前者的 29/30 次都是浪费。把轮询频率跟你关注的东西实际变化的频率对齐。

几个诊断命令要熟悉：`/usage` 看按 Skill、子代理、MCP 拆分的用量账单；`/goal` 不带参数看当前轮次和 token 消耗；`/workflows` 看每个 agent 分别烧了多少 token，也可以中途停掉任何 agent。

. . .

总结

四种循环的决策框架

文章末尾附了一张循环选择决策表，我把原文的逻辑翻译成了更直接的中文版：

你交出去的是	循环类型	适用场景	用什么工具
验证	轮次循环	探索、决策、一次性任务	验证 Skill

停止条件	目标循环	你知道「做完」长什么样	/goal
触发	时间循环	工作有规律地出现	/loop、/schedule
Prompt 本身	主动循环	工作重复且定义清晰	以上全部 + workflows

从上往下读，每往下一行，你交给系统的东西就多一层：先是验证那一步，然后是终点线怎么画，然后是触发器谁按，到底层连 Prompt 内容都模板化了。

这篇文章最有价值的判断是结尾那句：**跑在前面的工程师，优势不在写 Prompt，在設計循环。**

Prompt Engineering 的红利期正在过去。模型越来越强，Prompt 写得差一点它也能理解你的意思。但 Loop Engineering 的门槛不会降。怎么把模糊的任务拆成可验证的子目标、怎么设定机器可读的停止条件、怎么在自动化和成本之间找平衡，这些是纯工程判断，模型再强也替不了你。

如果你想开始用循环，从一个你目前是瓶颈的任务开始。问自己三个问题：

能把「做完」写成一个可验证的条件吗？目标能表达成数字或状态吗？这个工作有规律地出现吗？
答案会告诉你适合用哪种循环。

别急着上 /schedule 和 proactive。先把验证 Skill 写好，把 /goal 的停止条件设对。这两件事做好了，你已经比 90% 的 Claude Code 用户效率高了。

• • •

本文收录于「📖 Claude 官文精读」合集

• • •

觉得有收获？转发给也在用 Claude Code 的同事

关注「克劳德猎手」，获取更多深度解读



克劳德猎手🐾

Claude 猎手从不追热点，只追真相，读懂 AI 行业每一件大事，Anthropic 大模型独家深... >

194篇原创内容

公众号

👍 喜欢作者

“ You're absolutely right! ”

7 人付费 >



可赠送给一位朋友免费阅读

去赠送

 Claude 官文精读 · 目录 ≡

[< 上一篇 · Claude Code 动态工作流：六种 Agent 编排模式，让 AI 为每个任务写定制脚本](#)

作者提示: 个人观点，仅供参考

留言

写留言

